3: device drivers for console and display

Due by noon, Wed Sept 27, 2023

In this homework, we extend the existing console device driver to support colored text, and create a brand new device driver for graphics display. Provided are a couple of user-space programs that use the new (and as of yet non-existent) functionality to show stuff on the screen.

Getting started

Create your github turn-in repository using this link: https://classroom.github.com/a/h5k_dheN

After running "make qemu", you will see two new binaries **prettyprint** and **imshow** in a running xv6. Try to run them, neither program takes any arguments.

For this homework, we start with a slightly modified version of the xv6 original, available under the hw3 branch. Assuming you have configured the class repository as "origin" in git, first **git pull** the latest from the repository, then **git checkout origin/hw3 -b hw3** to check out the template into a local branch called hw3.

To learn about the difference between the master branch and the hw3 skeleton code, run **git diff origin/master *.{c,h,S}**. The template introduces a couple of new user programs, some VGA support functions, adds the ioctl system call, and makes a small structural change to how device drivers hook into the kernel.

Part 1: Console driver (50%)

With `prettyprint` ("pretty-print") a correct solution would display colorful text. Read `prettyprint.c` to see how it is intended to work.

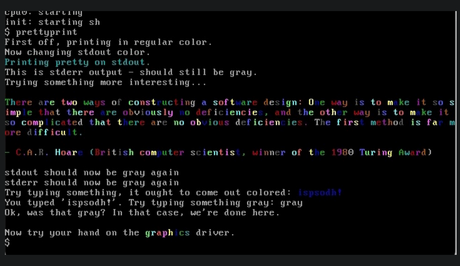
Here, `ioctl()` (read "I/O control") is a classic Unix system call that was added to xv6 for this assignment. The purpose of **ioctl**, is to support various configuration settings to I/O devices. In the Unix model, I/O devices are represented by device files, notably `console` and `display` in this assignment.

Hence, your job is to extend the functionality of the existing ioctl system call, to support the expected behavior of the `prettyprint` program. Remember the color configured by each call to ioctl, and make sure this color is used to print subsequent characters on the appropriate file descriptor. There is a separate ioctl mode for changing the "global" color, used when displaying input and cprintf output. Note that other output (say from cprintf, console input, or stderr), should not be affected by any color change to stdout, and vice versa. To help with this, the hw3 template includes changes to init.c, which opens /console a second time for stderr, rather than dup()ing stdout.

study tip: try to understand and follow the end-to-end life of a system call, how arguments are processed, how return values make it to the application, and so on. Read the console.c code and make sure you know what's going on. How do characters make it from a keyboard interrupt to a reading process? For a more interesting example, what happens if the user presses backspace while a program is reading from stdin?

The "driver" of the console is initialized in consoleinit(). You code for Part 1 mainly goes to console.c, plus some declarations in defs.h.

Below is the expected output.



Hints

In **struct file**, you will find a new field void* **dev_payload**. This field may be used by device drivers to store driver-specific information in the struct file. This could be a pointer to a struct holding a lot of information, but in our case, we just need an int to remember the color stored by ioctl.

Read pprint.c, console.c, and other related source code carefully. Don't rush to write code before understanding the expected behavior.

To output colored text, search the "gray on black" comment in `cgaputc()`. Try to experiment with printing a different color by editing that line of code.

How do you pass the color into that function? You need to change some function's prototype and/or add new functions to achieve it.

Debugging

When debugging, a good practice is to set the CPUS in Makefile to 1.

Just add the interested kernel function names to .gdbinit.tmpl:

```
...
b consoleinit
b consolewrite
c
```

If the kernel panic()-ed, it will print out a list of addresses. That is the stack backtrace of the failed execution. Each address is the return address of a function call. You can find the the corresponding line in kernel.asm, and the instruction **before** the address shoud be a "call" instruction.

Part 2: Graphics driver (50%)

With `imshow`, the by-now-familiar cover image should show up on the screen, change color, and disappear (return to text mode). For full points, make sure the original console text remains after the program finishes. (hint: switching video modes automatically clears the corresponding buffer).

While `console` supports both read and write, your `display` only needs to support write (for showing pixels). Look at how the console driver is implemented and figure out how to implement a display driver.

Note that due to the virtual memory mapping, the virtual address `0xA0000` no longer maps to the video buffer (0xA0000 becomes user-space memory). Instead, you can find the buffer at `KERNBASE+0xA0000`.

Create a flie `display.c` to hold the display driver code, plus "hooks" elsewhere. You'll need to implement driver initialization, writes to the display, and `ioctls` to change mode and modify palette colors. To change mode and palette colors, use the helper functions provided in `vga.h`. The Mode13 is the graphic mode.

Dig a little deeper

After executing `imshow` and returned to Mode 3, you will notice that `prettyprint` starts to show wrong colors. This is because the two VGA modes are supposed to use different palette settings. It's the driver's duty to configure the palette settings when switching modes. Your need to "fix the bug" by restoring the correct palette settings after switching to the VGA mode 3. **You only need to change vga.c to fix the bug.** Implement `cgaRestorePalette()`. You can get clues about how to do this by reading the functions around it.

Hints

The `display` device file is created for you upon startup, in init.c. For `sys_ioctl` and `sys_write`, try to mirror how the console device is implemented.

In addition to your new code in `display.c` (should be less than 50 lines), you will also need to add/change a few lines in `Makefile`, `main.c`, and `defs.h`.

You DON'T need to call setdefaultVGApalette().

Turn-in instructions

As in previous homeworks, commit your changes and push to github.

Don't forget to double check that your turn-in was successful, by cloning the entire repo to somewhere else, building, and testing.

hw3's new syscall -- ioctl()

A new syscall `ioctl()` is added to the hw3 branch.

- User space: `user.h` and `usys.S`
- Kernel space: `syscall.c` and `sysfile.c`
- `sys_ioctl()` -> `fileioctl()` -> `consoleioctl()` -> ?? what should be done here?

Read `fileioctl()` carefully to see what's the three arguments.

We use `printf` to print on the screen: (all user-space code)

- `printf()` at printf.c
- `putc()` at printf.c
- `write()` -- starts the syscall

At the kernel side:

- After some .S and dispatching code...
- `sys_write()` -> `filewrite()`

cross-reference fails again. function pointers!

grep 'devsw':

```
return devsw[f->ip->major].write(f, off, addr, n);
```

we want to see who set those function pointers:

- grep for "`write =`"
- `consoleinit()` in console.c
- This search is easy in xv6 because there is only one place where the "write" function pointer is assigned. It can be extremely painful to search in a big code base such like the Linux kernel.

Now we know filewrite() effectively calls consolewrite():

- `consolewrite()` -> `consputc()` -> `cgaputc()` -> ...
- `crt[pos++] = (c&0xff) | 0x0700;`

How to manipulate text/color in VGA Mode 3: https://wiki.osdev.org/Text_UI

Compare the arguments for `consolewrite()` and `consputc()`. Something is missing?